RAPPORT TECHNIQUE

Diagnostic de Défauts dans les Moteurs CC 12V

Classification par Apprentissage Automatique pour Déploiement sur ESP32

1. Introduction

1.1 Objectif du projet

Ce projet vise à développer un système de diagnostic automatique pour les petits moteurs à courant continu (CC) 12V. Le système doit être capable de classifier en temps réel l'état de fonctionnement du moteur en trois catégories :

- Normal : le moteur fonctionne dans ses conditions nominales
- Surcharge : une contrainte mécanique ou électrique excessive est détectée
- Court-circuit : un défaut électrique sévère compromet le fonctionnement du moteur

Le diagnostic repose sur l'analyse de signaux issus de trois capteurs :

1. Capteur de courant (ex. ACS712) — mesure du courant consommé
2. Capteur de tension (diviseur de tension) — mesure de la tension d'alimentation
3. Capteur de vitesse (encodeur ou capteur à effet Hall) — mesure de la vitesse de rotation

1.2 Contrainte matérielle

Le modèle de classification entraîné est destiné à être déployé sur un microcontrôleur ESP32, ce qui impose des contraintes strictes :

Contrainte	Implication
Mémoire RAM limitée (~520 Ko)	Le modèle doit avoir une empreinte mémoire de quelques Ko
Pas de GPU	Les calculs doivent se faire en CPU avec arithmétique flottante simple
Pas de bibliothèques ML	Le modèle doit être exporté en code C natif
Temps réel	L'inférence doit se faire en moins de 100 ms

Ces contraintes excluent les architectures complexes (réseaux de neurones profonds, CNN, LSTM) et orientent le choix vers des algorithmes classiques de machine learning.

2. Analyse du Dataset

2.1 Présentation du dataset ITSC

Le dataset utilisé est le ITSC Dataset (Inter-Turn Short-Circuit), développé par le Laboratoire de Génie Électrique de l'Université de Guanajuato (Mexique). Il est disponible publiquement sur GitHub : https://github.com/ibarram/ITSC

Ce dataset contient des signaux de courant triphasé mesurés sur un moteur à induction à cage d'écureuil (Baldor CM3542, 0.75 HP, 230 VAC, 1725 RPM). Le moteur a été instrumenté avec des commutateurs permettant de simuler des défauts de court-circuit inter-spires à différents niveaux de sévérité.

Référence scientifique : Cardenas-Cornejo, J.-J. et al., "Classification of inter-turn short-circuit faults in induction motors based on quaternion analysis", Measurement, Volume 222, 2023.

2.2 Structure du dataset

Le dataset est fourni en trois versions :

Dossier	Contenu	Dimensions par fichier	Total
RAW_Signals	Signaux bruts non traités, tous dans un seul dossier	5000 × 3	65 fichiers
RAW_Signals_SF	Mêmes signaux bruts, organisés par sous-dossiers de classe	5000 × 3	13 dossiers × 5 fichiers
Cropped_Signals_SF	Signaux pré-traités et recadrés, par sous-dossiers	1000 × 3	13 dossiers × 5 fichiers

Des fichiers MATLAB (.mat) sont également fournis (Data_ITSC.mat et RAWData_ITSC.mat).

Pour ce projet, nous utilisons Cropped_Signals_SF car :

- Les signaux sont déjà filtrés et centrés sur l'état stationnaire du défaut
- La dimension réduite (1000 au lieu de 5000 échantillons) est plus adaptée au traitement ESP32
- L'organisation par sous-dossiers facilite le chargement et l'attribution des classes

2.3 Description des classes originales

Chaque fichier CSV contient une matrice de 1000 lignes × 3 colonnes, où :

- Les 1000 lignes représentent des points temporels (1 seconde à 1 kHz)
- Les 3 colonnes correspondent aux 3 phases de courant (A, B, C) du moteur

Le dataset comporte 13 classes, soit 65 fichiers au total :

Classe	Description	Nb fichiers
SC_HLT	État sain (Healthy) — aucun défaut	5
SC_A1_B0_C0	Court-circuit phase A à 10%	5
SC_A2_B0_C0	Court-circuit phase A à 20%	5
SC_A3_B0_C0	Court-circuit phase A à 30%	5
SC_A4_B0_C0	Court-circuit phase A à 40%	5
SC_A0_B1_C0	Court-circuit phase B à 10%	5
SC_A0_B2_C0	Court-circuit phase B à 20%	5
SC_A0_B3_C0	Court-circuit phase B à 30%	5
SC_A0_B4_C0	Court-circuit phase B à 40%	5
SC_A0_B0_C1	Court-circuit phase C à 10%	5
SC_A0_B0_C2	Court-circuit phase C à 20%	5
SC_A0_B0_C3	Court-circuit phase C à 30%	5
SC_A0_B0_C4	Court-circuit phase C à 40%	5

Convention de nommage : SC_AX_BX_CX_00Y où X est le niveau de défaut (0-4) pour chaque phase et Y le numéro de répétition (1-5).

2.4 Acquisition des données

Les mesures ont été réalisées avec :

- Capteurs : Transformateurs de courant split-core SCT013
- Module d'acquisition : NI-9215 Series
- Logiciel : LabVIEW
- Conditions : État stationnaire, sans charge, à 60 Hz
- Fréquence d'échantillonnage : 1 kHz
- Durée d'acquisition : 5 secondes (signaux bruts), recadrés à 1 seconde

2.5 Adaptation au problème à 3 classes

Le dataset original comporte 13 classes, mais notre problème nécessite seulement 3 classes. Le remapping suivant a été défini :

Classe cible	Classes ITSC sources	Nb fichiers	Justification
Normal	SC_HLT	5	État sain, fonctionnement nominal
Surcharge	Niveaux 1-2 (10%-20%) des 3 phases	30	Les défauts légers produisent des signatures similaires à une surcharge mécanique
Court-circuit	Niveaux 3-4 (30%-40%) des 3 phases	30	Les défauts sévères correspondent à un court-circuit franc

Justification du regroupement :

- Les défauts inter-spires à faible niveau (10-20%) se manifestent par une légère augmentation du courant et une distorsion mineure du signal, comportement analogue à une surcharge
- Les défauts à niveau élevé (30-40%) provoquent des perturbations majeures du signal, caractéristiques d'un court-circuit
- Ce regroupement permet d'obtenir des classes équilibrées (Surcharge et Court-circuit ont 30 échantillons chacun), même si Normal est sous-représenté (5 échantillons)

2.6 Correspondance des colonnes aux capteurs

Les 3 colonnes du dataset (phases A, B, C) sont mappées aux 3 capteurs du système embarqué :

Colonne CSV	Phase ITSC	Capteur ESP32
Colonne 0	Phase A	Courant
Colonne 1	Phase B	Tension
Colonne 2	Phase C	Vitesse

Ce mapping est cohérent car le modèle apprend les relations entre 3 signaux simultanés, quelle que soit leur nature physique exacte.

3. Extraction des caractéristiques (Feature Engineering)

3.1 Pourquoi extraire des caractéristiques ?

Les signaux bruts (1000 × 3 = 3000 valeurs par échantillon) ne peuvent pas être utilisés directement comme entrée du modèle sur ESP32, car :

1. La dimension est trop élevée pour une inférence rapide
2. Les algorithmes classiques (arbres, SVM) ne traitent pas efficacement les séries temporelles
3. Les caractéristiques statistiques capturent l'essence du signal en un vecteur compact

3.2 Caractéristiques extraites

Pour chaque capteur (3 au total), on extrait 15 caractéristiques, soit 45 au total.

Caractéristiques temporelles (10 par capteur)

#	Caractéristique	Formule	Signification physique
1	Moyenne	$\mu = (1/N)\ \Sigma x_i$	Composante continue du signal
2	Écart-type	$\sigma = \sqrt{[(1/N)\ \Sigma(x_i-\mu)^2]}$	Dispersion autour de la moyenne
3	Valeur RMS	$\sqrt{[(1/N)\ \Sigma x_i^2]}$	Valeur efficace (puissance du signal)
4	Maximum	max(x)	Pic maximal du signal
5	Minimum	min(x)	Creux minimal du signal
6	Amplitude crête-à-crête	max(x) - min(x)	Étendue totale du signal
7	Kurtosis	$E[(x-\mu)^4]/\sigma^4 - 3$	Présence de pics ou de valeurs extrêmes
8	Asymétrie (Skewness)	$E[(x-\mu)^3]/\sigma^3$	Asymétrie de la distribution
9	MAD	$(1/N)\ \Sigma\lvert x_i - x_{i-1}\rvert$	Rugosité/variabilité du signal
10	IQR	$Q_{75} - Q_{25}$	Dispersion des valeurs centrales

Caractéristiques fréquentielles (5 par capteur)

#	Caractéristique	Description
11	Énergie spectrale	Somme des carrés du spectre FFT
12	Fréquence dominante	Fréquence ayant la plus grande amplitude
13	Fréquence moyenne pondérée	Barycentre fréquentiel
14	Dispersion spectrale	Écart-type des amplitudes spectrales
15	Ratio du pic spectral	Rapport entre le pic maximal et la somme totale

3.3 Pourquoi ces caractéristiques ?

- Caractéristiques temporelles : elles capturent les modifications d'amplitude, de forme et de distribution du signal causées par les défauts. Un court-circuit augmente le RMS et le peak-to-peak, tandis qu'une surcharge modifie la kurtosis et l'asymétrie.
- Caractéristiques fréquentielles : les défauts introduisent des harmoniques supplémentaires dans le spectre. La fréquence dominante et l'énergie spectrale sont des indicateurs fiables de l'état du moteur.

- Calculables sur ESP32 : toutes ces caractéristiques ne nécessitent que des opérations arithmétiques simples (addition, multiplication, comparaison) facilement implémentables en C.

4. Préparation des données

4.1 Partitionnement

Les données sont divisées en :

- 80% pour l'entraînement (52 échantillons)
- 20% pour le test (13 échantillons)

Le partitionnement est stratifié, ce qui garantit que la proportion de chaque classe est conservée dans les deux ensembles.

4.2 Normalisation

Les caractéristiques sont normalisées avec le StandardScaler de scikit-learn, qui transforme chaque caractéristique pour avoir une moyenne de 0 et un écart-type de 1 :

```
x_normalisé = (x - moyenne) / écart-type
```

La normalisation est indispensable car :

- Les caractéristiques ont des échelles très différentes (l'énergie spectrale est de l'ordre de 10^5, la fréquence dominante est de l'ordre de 10^1)
- Les algorithmes comme KNN, SVM et MLP sont sensibles à l'échelle des données
- Les paramètres du scaler (moyenne et écart-type par caractéristique) sont exportés dans scaler_params.h pour être reproduits sur ESP32

5. Modèles de classification

5.1 Choix des algorithmes

Huit algorithmes ont été évalués, choisis pour leur compatibilité avec les contraintes ESP32 :

Algorithme	Principe	Avantage ESP32	Limitation
Decision Tree	Séquence de tests sur les caractéristiques	Très compact (quelques Ko), rapide	Peut sur-apprendre
Random Forest	Ensemble de 10 arbres de décision	Plus robuste qu'un seul arbre	Taille proportionnelle au nb d'arbres

KNN	Comparaison aux k voisins les plus proches	Pas d'entraînement	Nécessite de stocker toutes les données
SVM linéaire	Hyperplan séparateur de marge maximale	Très compact (poids + biais)	Limité aux frontières linéaires
Régression logistique	Modèle linéaire probabiliste	Le plus compact possible	Limité aux frontières linéaires
Perceptron multicouche	Réseau de neurones à 1-2 couches cachées	Taille contrôlable	Plus lent à inférer
HistGradientBoosting	Boosting avec histogrammes	Très performant, relativement compact	Plus complexe qu'un arbre unique
XGBoost	Extreme Gradient Boosting	État de l'art pour données tabulaires	Taille du modèle variable

5.2 Optimisation des hyperparamètres

Chaque modèle est optimisé par recherche exhaustive sur grille (GridSearchCV) avec validation croisée stratifiée à 5 plis.

Le processus pour chaque modèle :

4. Définir une grille d'hyperparamètres à tester
5. Pour chaque combinaison, entraîner sur 4 plis et évaluer sur le 5ème
6. Répéter sur les 5 rotations possibles
7. Sélectionner la combinaison ayant la meilleure accuracy moyenne
8. Réentraîner le modèle final sur l'ensemble d'entraînement complet

5.3 Métriques d'évaluation

Les modèles sont évalués selon :

- Accuracy : proportion de prédictions correctes
- F1-Score pondéré : moyenne harmonique de la précision et du rappel, pondérée par la taille de chaque classe
- Matrice de confusion : visualisation des erreurs de classification par classe
- Rapport de classification : précision, rappel et F1-score par classe

6. Exportation pour ESP32

6.1 Conversion en code C

Le meilleur modèle compatible ESP32 est converti en code C natif à l'aide de la bibliothèque m2cgen (Model to Code Generator). Cette bibliothèque traduit les modèles scikit-learn en une fonction C pure score() qui :

- Ne dépend d'aucune bibliothèque externe
- Est directement compilable avec le toolchain Arduino/ESP-IDF
- Accepte un tableau de flottants (les 45 caractéristiques normalisées) en entrée
- Renvoie un tableau de 3 scores (un par classe)

6.2 Fichiers générés

Fichier	Contenu
model_esp32.c	Code C du modèle de classification (fonction score())
scaler_params.h	Tableaux de moyennes et d'écarts-types pour la normalisation

6.3 Intégration sur ESP32

L'intégration sur ESP32 suit le pipeline :

9. Acquisition des 3 signaux (1000 échantillons à 1 kHz)
10. Calcul des 45 caractéristiques en C (même code que la fonction Python extract_features)
11. Normalisation avec les paramètres de scaler_params.h
12. Appel de la fonction score() de model_esp32.c
13. Sélection de la classe par argmax du tableau de sortie

7. Conclusion

Ce travail a permis de :

14. Analyser et restructurer le dataset ITSC pour un problème de classification à 3 classes
15. Extraire 45 caractéristiques pertinentes (temporelles et fréquentielles) de chaque signal
16. Entraîner et comparer 8 algorithmes de classification avec optimisation par validation croisée
17. Exporter le meilleur modèle en code C directement exécutable sur ESP32

Le système proposé permet une détection des défauts moteur en temps réel avec une empreinte mémoire minimale, adapté aux microcontrôleurs basse consommation.